

ISW4 — Proyecto Final

Version 1 — Arquitectura Monolitica MVC

Informe Tecnico

Integrantes:

Maria Jose Betancourt - A00403987, Joshua Sayur - A00404707, Martín Borrero Herrera - A00403871, Javier Andrés Paz Luna - A00405619, Camilo Andrés Martínez Moreno - A00405205

1. Objetivo

Calcular la velocidad promedio por ruta por mes usando los registros de posicion GPS (datagrams) de los buses del SITM-MIO de Cali. La solucion implementa una arquitectura monolitica limpia basada en el patron MVC.

2. Como se hace el calculo

2.1 Datos de entrada

- datagrams-MiniPilot.csv / datagrams4Pilot.csv: registros GPS de cada bus.
- lines-241-ActiveGT.csv: lista de rutas activas del piloto.

2.2 Columnas utilizadas del CSV de datagrams

Indice	Campo	Descripcion
[4]	latitude	Latitud GPS — entero, dividir por 1e7 para obtener grados decimales
[5]	longitude	Longitud GPS — entero, dividir por 1e7 para obtener grados decimales
[7]	lineId	ID de la ruta asignada al bus en el IVU
[10]	datagramDate	Fecha y hora del evento (yyyy-MM-dd HH:mm:ss)
[11]	busId	ID del bus en el sistema IVU

2.3 Algoritmo de calculo

- Pre-filtrar: se descartan datagrams con lineId <= 0 o cuya ruta no este en la lista activa.
- Ordenar: los datagrams se ordenan por busId -> lineId -> datagramDate para garantizar pares consecutivos coherentes.
- Iterar pares: para cada par consecutivo del mismo bus y misma ruta:
 - Se calcula la distancia entre los dos puntos GPS con la formula Haversine.
 - Se calcula el tiempo transcurrido en horas entre ambas fechas.

- Velocidad = Distancia (km) / Tiempo (h). Se descarta si supera 80 km/h (outlier) o si distancia/tiempo son ≤ 0 .
- Agrupar: cada velocidad valida se acumula en un DoubleSummaryStatistics identificado por lineId|yyyy-MM.
- Promediar: se calcula el promedio de cada grupo y se genera un SpeedRecord por rutas.

2.4 Formula Haversine

```
a = sin2(Delta_lat/2) + cos(lat1) * cos(lat2) * sin2(Delta_lon/2)
distancia = 6371 * 2 * atan2(sqrt(a), sqrt(1-a)) [km]
```

Donde lat1, lat2, lon1, lon2 estan en radianes. Esta formula calcula la distancia del camino mas corto sobre la superficie de la esfera terrestre.

3. Optimizaciones aplicadas (V1)

Las siguientes mejoras se aplicaron manteniendo la arquitectura monolitica secuencial, sin concurrencia:

Optimizacion	Descripcion
Pre-filtrado temprano	Se eliminan datagrams invalidos antes del sort, reduciendo el volumen de datos a ordenar.
Iteracion unica (single-pass)	El for principal recorre la lista una sola vez con un puntero 'previous', eliminando el acceso doble por indice i-1.
DoubleSummaryStatistics	En vez de List<Double> por grupo (que acumula todos los valores), se usa DoubleSummaryStatistics que mantiene count y sum en $O(1)$ por insercion, usando mucha menos memoria.
Haversine optimizada	Se precalcula $TO_RAD = \pi/180$ como constante estatica para evitar la division en cada llamada.
HashMap con capacidad inicial	Se inicializa con capacidad 256 para evitar rehashing durante la acumulacion de grupos.
HashSet para rutas activas	El lookup de rutas activas es $O(1)$ con HashSet en vez de $O(n)$ iterando la lista en cada datagram.

4. Resultados del experimento

Tiempos de ejecución medidos en la maquina local.

Dataset	Carga (ms)	Calculo (ms)	Total (ms)	Registros
datagrams-MiniPilot.csv	7774	4474	13045	97

Los tiempos se imprimen automaticamente en consola al finalizar la ejecucion:

```
Carga de datos : XXX ms
Calculo         : XXX ms
```

Total	: XXX ms
Registros	: XXX

5. Arquitectura MVC

Capa	Clase(s)	Responsabilidad
Model	Datagram, Route, SpeedRecord	Entidades del dominio
Repository	DatagramRepository, RouteRepository	Lectura de archivos CSV
Service	SpeedCalculationService	Logica de negocio: filtrar, ordenar, calcular, agrupar
View	ExcelView	Presentacion: exportacion Excel
Controller	ProcessingController	Coordinacion y medicion de tiempos